Frontend Engineering Playbook & Standards

This document outlines the strict engineering conventions, patterns, and rules for the nextBooq frontend team. Any pull request that violates these principles should be rejected.

1. Authentication & Security Handling

We use a strict **HttpOnly Cookie** approach. The frontend code is entirely blind to the actual JWT.

- **Rule 1: Never store tokens.** Do not use `localStorage`, `sessionStorage`, or Zustand to store the auth token.
- **Rule 2: Axios Credentials.** The Axios instance (`src/hooks/api/axios-instance.ts`) must have `withCredentials: true` set globally so that cookies are automatically attached to every request.
- **Rule 3: The `useAuth` Hook.** The global source of truth for user state is a React Query hook that hits `GET /api/v1/auth/me`.
- **Rule 4: Client-Side RBAC.** Protect UI elements using the `staff_memberships` array returned by `useAuth`.

```
// Example: Hiding the Settings button from a Host
const { user } = useAuth();
const hasPermission = user.staff_memberships[0].permissions.includes('edit_se
if (!hasPermission) return null;
```

2. API Communication & React Query Rules

TanStack Query (React Query) manages all asynchronous state.

- **Query Key Factory:** All query keys must be strictly typed and centralized to prevent typos during cache invalidation.
  - *Good:* `queryKey: ['reservations', cafeId, date]`
  - *Bad:* `queryKey: ['getReservations', id]`
- **Stale Times:** By default, React Query marks data as stale instantly.
  - *Cafe Details (`/cafes/:slug`):* Set `staleTime: 1000 * 60 * 60` (1 hour). This data rarely changes.
  - *Live Availability & Manifest:* Leave `staleTime: 0`. We always want the freshest table data.
- **Mutations & Invalidation:** Every successful mutation (POST, PUT, DELETE) must invalidate the relevant queries.

```
// After a successful PUT to update table status:
onSuccess: () => {
  queryClient.invalidateQueries({ queryKey: ['reservations', cafeId, today] })
}
```

3. Global Error Handling (Axios Interceptors)

Individual components should not be responsible for catching standard HTTP errors (like 401s or 500s). This must be handled globally to ensure a consistent UX.

Configure an Axios Interceptor in `axios-instance.ts`:

- **401 Unauthorized:** Instantly call `queryClient.setQueryData(['me'], null)` and redirect the user to `/auth/login`.
- **403 Forbidden:** Display a generic toast: *"You do not have permission to perform this action."*
- **409 Conflict (Crucial for OCC):** If a 409 is caught during a booking or status update, display a specific toast: *"This table state has changed. Please refresh to see the latest updates."*
- **422 Unprocessable Entity:** Map backend validation errors directly to React Hook Form fields.
- **500 Internal Error:** Display a generic fallback toast: *"Something went wrong on our end. Please try again."*

4. UI Component & Styling Standards

We use **Tailwind CSS** and **shadcn/ui**.

- **The Single Source of Truth:** Do not write custom CSS files. If a style cannot be achieved with standard Tailwind classes, extend the Tailwind theme in `tailwind.config.ts`.
- **Component Purity:** Components inside `src/components/ui` (like Buttons, Inputs, Modals) must be "dumb". They accept props and emit events. They **must never** contain React Query hooks or Axios calls.
- **Multi-Tenant Theming:** Do not hardcode brand colors (e.g., `text-blue-500`). Use CSS variables injected by the `TenantThemeProvider` (e.g., `text-primary`).

```
/* globals.css */
:root {
  --primary: 222.2 47.4% 11.2%; /* Default NextBooq Blue */
}
.theme-centralperk {
  --primary: 142.1 76.2% 36.3%; /* Central Perk Green */
}
```

5. Form Management

- **Rule:** Every form in the application must use **React Hook Form** paired with **Zod** for schema validation.
- **Validation Parity:** The Zod schemas on the frontend must exactly mirror the validation rules on the Express backend (e.g., ensuring `party_size` is a positive integer, passwords meet complexity requirements).

6. Real-Time Dashboard (WebSockets)

- **Connection Lifecycle:** The WebSocket connection (`socket.io-client`) should only be initialized when a user mounts the `(dashboard)/layout.tsx` component. It must be cleanly disconnected on unmount to prevent memory leaks.
- **Event Handling:** WebSocket events should NOT update the UI directly. They should trigger a React Query background refetch.

```
socket.on('reservation_updated', () => {
   // Silently updates the manifest without disrupting the user's current act
   queryClient.invalidateQueries({ queryKey: ['reservations'] });
});
```